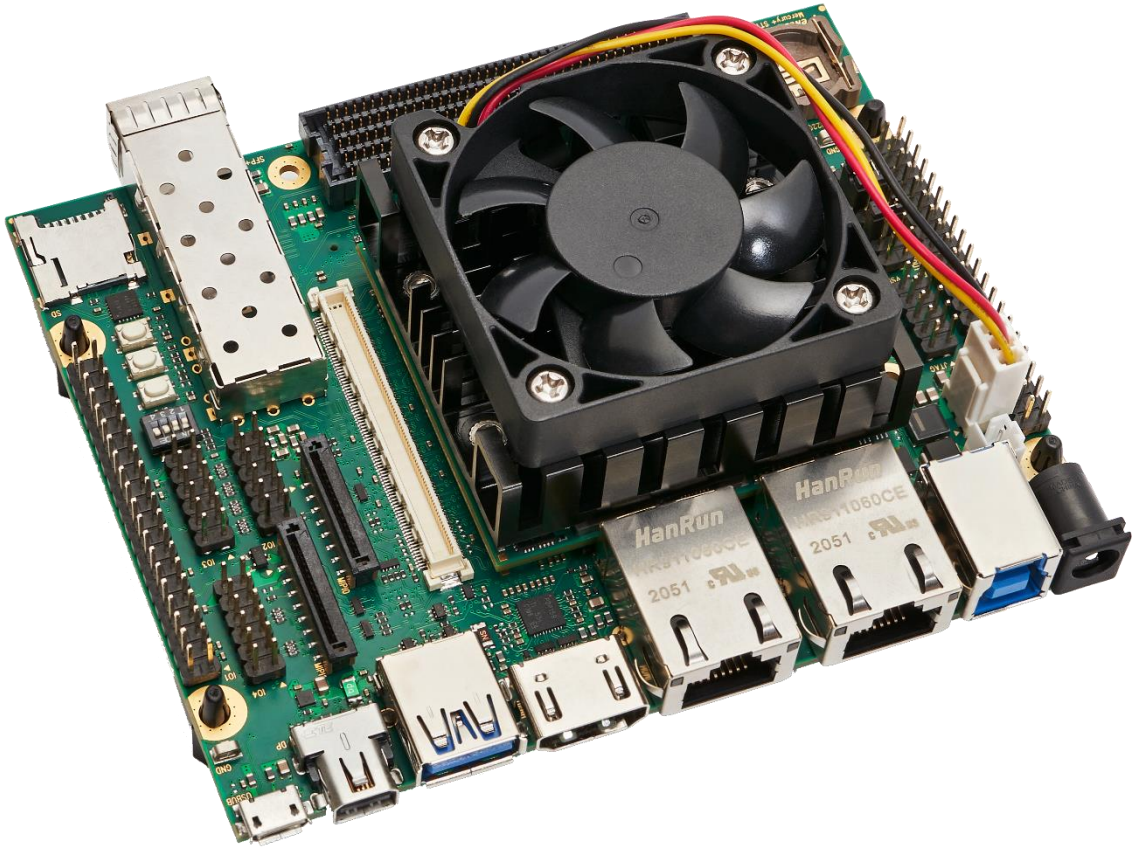
Design-In-Kit
Mercury XU5

User Manual

AI Face Detection & Image Classification



Enclustra GmbH – Räffelstrasse 28 – CH-8045 Zürich – Switzerland
Phone +41 43 343 39 43 – www.enclustra.com

Purpose

The purpose of this document is to help the user speed-up the design-in process of an Enclustra System-on-Module and shorten time-to-market.

Summary

This document first gives an overview of the Mercury XU5 Design-in kit followed by a detailed description of its set-up and building of the provided demos. In addition, references to other useful documents are included.

Product Information	Code	Name
Product	QSK-DK-ME-XU5-4EV-ST1	Design-in kit XU5

Document Information	Reference	Version	Date
Reference / Version / Date	D-0000-000-000	RC01	26.07.2021

Approval Information	Name	Position	Date
Written by	TVOE	Design Engineer	
Verified by	JDRA	Project Manager	
Approved by	DIUN	Manager, BU SP	

Copyright Reminder

Document History

Version	Date	Author	Comment
RC01		TVOE	First release

Table of Contents

1 Introduction

The Enclustra Design-in kit is a ready to use platform including 2 Xilinx Vitis AI based demos: face detection and image classification. The kit helps to cut development time for any Xilinx Zynq UltraScale+ MPSoC based application. To shorten time-to-market as much as possible, Enclustra offers broad design-in support for their products and a comprehensive ecosystem, offering all required hardware, software and support materials. Detailed documentation and reference designs make it easy to get started. A user manual, user schematics, a 3D-model, PCB footprints and differential I/O length tables and the Linux-based Board Support Package (BSP) are all provided. In combination with ready-made base boards and heatsinks, developing an FPGA project has never been easier.

1.1 Scope of This Document

This user manual explains all the steps to build and run the Xilinx Vitis-AI face detection and image classification demos based on ResNet50 and Xilinx Vitis on the Enclustra Mercury XU5 Design-in kit. The demos are derived from the Xilinx Vitis AI demos on Github [2].

1.2 Requirements

1.2.1 Hardware

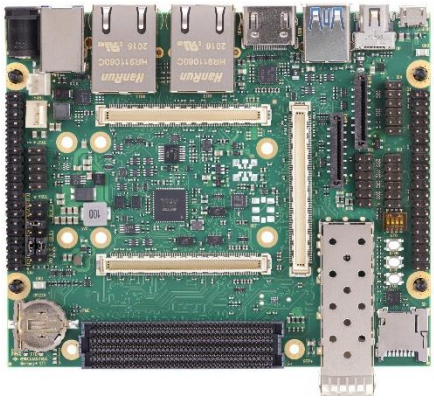
- Enclustra Mercury XU5-4/5EV SoC module
- Enclustra Mercury ST1 base board
- Heat sink and fan for Mercury XU5
- Display Port monitor (capable of 640x480@30fps)
- Logitech USB camera
- Micro-SD card
- Micro-SD card reader
- Mini DisplayPort to DisplayPort cable
- Micro USB cable
- optional: Network cable

1.2.2 Software

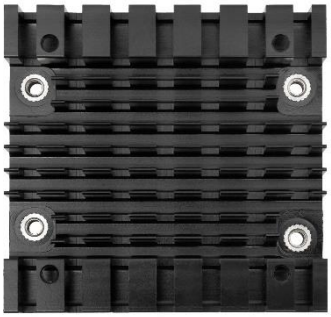
- Ubuntu 18.04
- PetaLinux 2020.2 and Vitis 2020.2
- GStreamer1.0
- A serial terminal (e.g. tio or tera term)

1.2.3 What's Inside the Box

Mercury XU5 Module



Mercury+ ST1 Base Board



Heatsink

Mounting Screws

Gap Pad

Fan

Power Supply & USB Cable

Web Cam

Micro SD Card

DisplayPort Cable

2 Creating the Vivado Project

The base reference design for the Mercury XU5 & Mercury+ ST1 from the Enclustra Github repository is used as a starting point [4] to build the demos. TCL scripts and a wrapper makefile were added to download the Xilinx Vitis-AI DPU core and add the core to the Enclustra reference design.

Note: There are two flows, the Vivado DPU flow and the Vitis DPU flow. Only the classic Vivado flow based on the according Xilinx TRD [5] is shown.

Note: The Vivado design can be built for other Mercury XU5 modules too. This can be achieved by setting the variable *MODULE_NAME* in the Makefile located in **`{<base_dir>}/reference_design`**. A possible variant would be ***ME-XU5-5EV-2I-D12E***

2.1 Build Steps

Step	Description
1	Download and extract the design files [8]: Enclustra Mercury XU5 Design-In Kit Sources
2	Source the Xilinx 2020.2 tools: **`source /opt/Xilinx/Vitis/2020.2/settings64.sh`** **Note**: Adapt the path in case the Xilinx tools are located in a different location/path.
3	Change the working directory to: **`{<base_dir>}/reference_design`** and create the base project: **`{<base_dir>}/reference_design$ make create`**
4	Download the DPU from the Xilinx Vitis AI Github repository using the provided Makefile target: **`{<base_dir>}/reference_design$ make dow_dpu`** This step will download the Vitis AI 1.3.1 tar file and extract it to the dow folder. A link to the edge DPU core will be placed into the ip_repo folder which is the default local IP core repository.
5	Add the DPU core to the Enclustra reference design: **`{<base_dir>}/reference_design$ make add_dpu`**
6	Build the bitstream: **`{<base_dir>}/reference_design$ make bitstream`** This generates the PL bit file.

Step	Description
7	Export the xsa file: **{<base_dir>}/reference_design$ make xsa** The XSA file is the handshake file between the PetaLinux and the Vivado project. If successful, the file is located in: **{<base_dir>}/reference_design/Vivado/ME-XU5-4EV-2ID12E/Mercury_XU5_ST1.xsa**

Table 1: FPGA Bitstream Generation – Step-By-Step Guide

.

2.2 Block Design

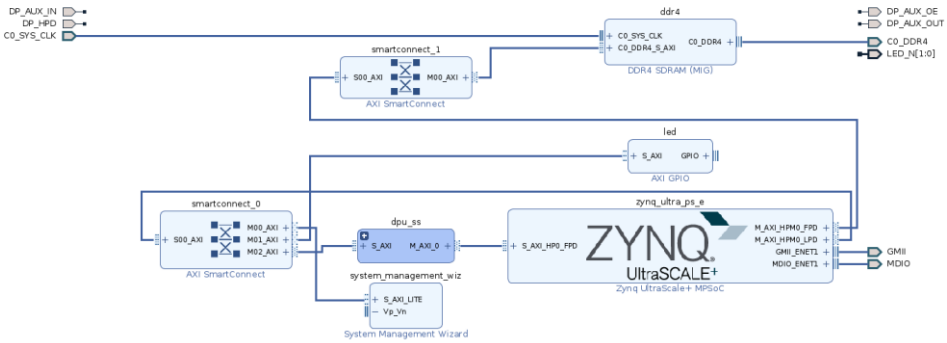


Figure 1: Top Level Block Design

For illustration purpose the Vivado design can be opened in GUI mode. The difference to the Enclustra base reference design is the changed dpu_ss (marked blue in Figure 1) hierarchical:

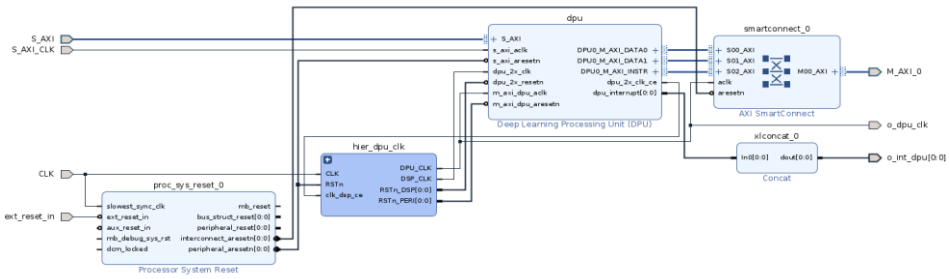


Figure 2: DPU subsystem

The DPU subsystem contains a dedicated clock manager which produces the DPU clock (with the same clock domain as the subsequent smartconnect and the S_AXI_HP0_FPD blocks) as well as the DPU DSP clock. The frequency of the DSP clock is twice the DPU clock. The current configuration uses 300 MHz

and 600 MHz. For this demo one DPU core using the B4096 (maximum) configuration is used. For applications where two networks are running concurrently, a configuration with two B2304 might be considered.

For further information on possible DPU configuration refer to [6].

3 Building the PetaLinux BSP

The Enclustra base PetaLinux Boards Support Package (BSP) is used as starting point for this demo. The following changes have been made:

- Add the DPU definitions to the device tree
- Added a separate meta layer (meta-vai-demo) containing:
  - DPU kernel drivers
  - libvitis-ai-2020.2-r1.3.2 yocto recipe (this recipe pulls the precompiled Vitis AI libraries)
  - Example applications facedetect and resnet50

3.1 Build Steps

Step	Description
1	Source the Xilinx 2020.2 tools: **source /opt/Xilinx/petalinux/2020.2/settings.sh**
2	Create the PetaLinux project from the provided .bsp file: **petalinux-create -t project -s {<path_to_bsp_dir>}/ME-XU5-4EV-2I-D12E.bsp --name ME-XU5-4EV-2I-D12E**
3	Change the working directory: **cd ME-XU5-4EV-2I-D12E**
4	Import the Vivado XSA file built in chapter 2.1: **petalinux-config --get-hw-description ={<base_dir>}/reference_design/Vivado/ME-XU5-4EV-2I-D12E --silentconfig**
4	Build the PetaLinux project: **petalinux-build**
5	Generate the boot files and the linux root filesystem: **petalinux-package --boot --fsbl --fpga --atf --pmufw --u-boot --force**
6	Prepare the SD Card with the file generated in images/linux of the PetaLinux project: 1. Copy the BOOT.BIN, system.dtb, boot.scr and Image files to the first FAT32 formatted partition of the SD Card. 2. Extract the rootfs.tar.gz file to the 2nd ext4 partition Instructions for partitioning and formatting an SD Card are described in Appendix H of the PetaLinux user guide [7]

Table 2: PetaLinux build step-by-step guide

For further information regarding the Xilinx PetaLinux build system refer to the user guide [7].

3.2 PetaLinux Directory Structure

```
project-spec/
├── attributes
├── configs
├── meta-user
│   ├── conf
│   │   ├── layer.conf
│   │   ├── petalinuxbsp.conf
│   │   └── user-rootfsconfig
│   ├── COPYING.MIT
│   ├── README
│   ├── recipes-apps
│   ├── recipes-bsp
│   │   ├── device-tree
│   │   │   ├── device-tree.bbappend
│   │   │   └── files
│   │   │   ├── dpu.dtsi
│   │   │   └── zynqmp_enclustra_mercury_xu5.dtsi
.
.
└── meta-vai-demo
├── conf
│   └── layer.conf
├── recipes-ai
│   ├── dpcma
│   ├── dpu
│   ├── libvitis-ai-2020.2-r1.3.2
│   │   └── libvitis-ai-2020.2-r1.3.2.bb
│   └── opencv
│   └── opencv_3.4.3.bbappend
├── recipes-apps
│   ├── facedetect
│   ├── resnet50
│   └── xlnx-vart-resnet50
```

A separate meta layer was added to the base BSP containing the drivers (configured as loadable kernel modules), the Vitis AI libraries as well as the demo application resnet50 and facedetect. For a quick test, the modified Xilinx app xlnx-vart-resnet50 was added too.

The differences to the Enclustra base PetaLinux BSP are described in the following table:

Path	Description
project-spec/meta-user/recipes-bsp/device-tree/dpu.dtsi	The device tree node of the DPU is not auto generated. This is the reason why a hand written overlay has to be added.

Path	Description
project-spec/meta-vai-demo/recipes-ai/dpcma/	Contains the dma driver used by the DPU
project-spec/meta-vai-demo/recipes-ai/dpu/	Contains the DPU Linux kernel driver
project-spec/meta-vai-demo/recipes-ai/libvitis-ai-2020.2-r1.3.2	Recipe pulls the precompiled Vitis AI user space libraries from a Xilinx server. The libraries provide the high level API for the C/C++ application.
project-spec/meta-vai-demo/recipes-ai/opencv	Overlay to compile OpenCV with GStreamer support
project-spec/meta-vai-demo/recipes-ai/apps	Contains the demo C/C++ source files as well as the CNN models compiled for the DPU configuration.

Table 3: PetaLinux meta-layer description

4 Demo Application Design

The demo applications facedetect and resnet50 use the same basic application structure. The application is split into an image capture thread, a DPU processing thread and an image output thread. The software uses OpenCV functions to fetching images from a Video for Linux device (/dev/videox) and for manipulating them (scaling, text overlay, bounding boxes, color space conversion). To keep the application simple none of these tasks were outsourced to the programmable logic of the Xilinx Zynq UltraScale+ MPSoC device. For the same reason, an USB camera is used: this simplifies the image pipeline setup. The limitation for the frame rate is given by the USB camera (30fps).

The applications are linked to the Vitis AI runtime libraries, which are necessary for the DPU management.

The DPU management code as well as pre- and post-processing functions were reused from the Xilinx Vitis AI examples, available on github [2]:

- Vitis-AI/demo/VART/
- Vitis-AI/demo/Vitis-AI-Library/samples/

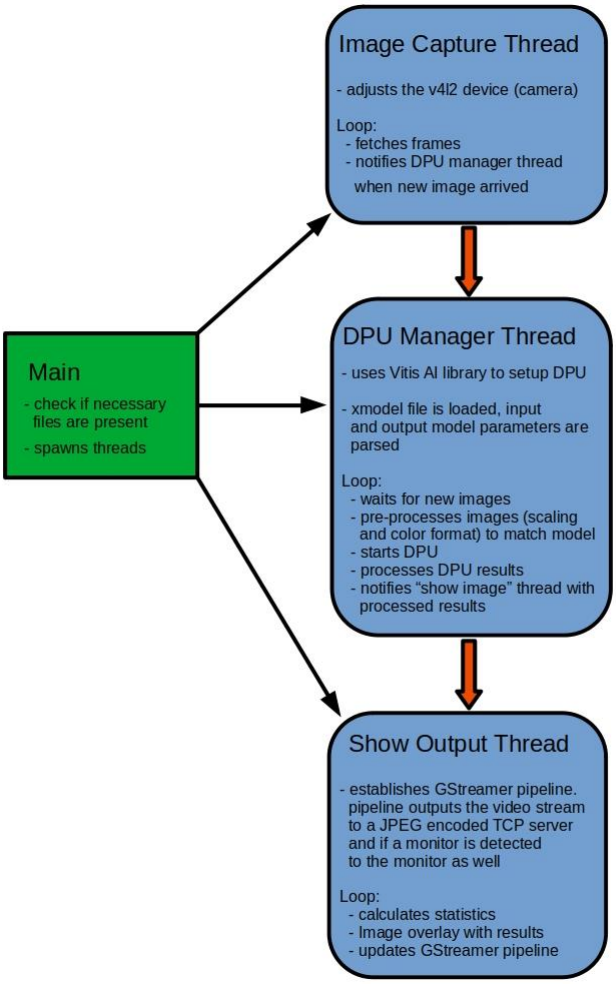
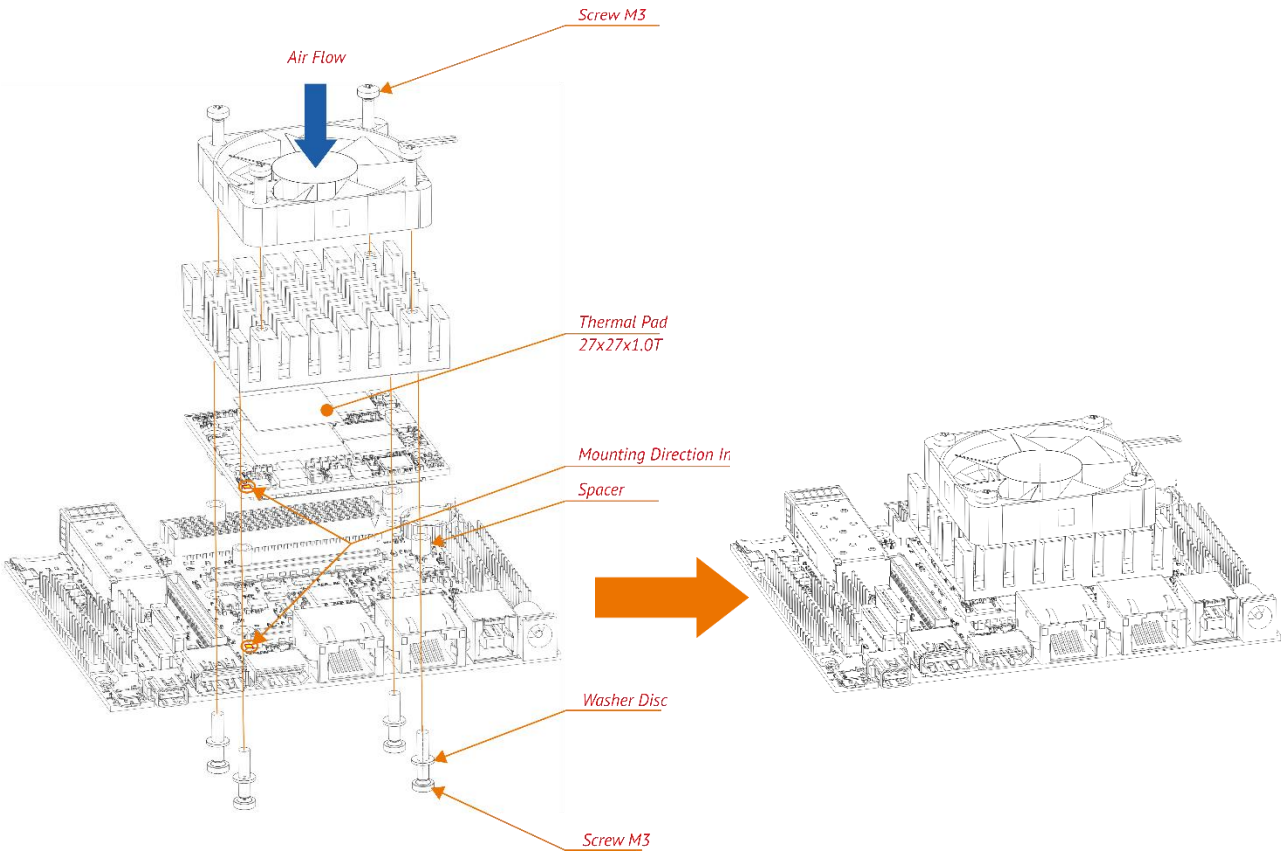


Figure 3: Software architecture

The source code of all demo applications is part of the provided PetaLinux BSP, in the folder **project-spec/meta-vai-demo/recipes-apps**

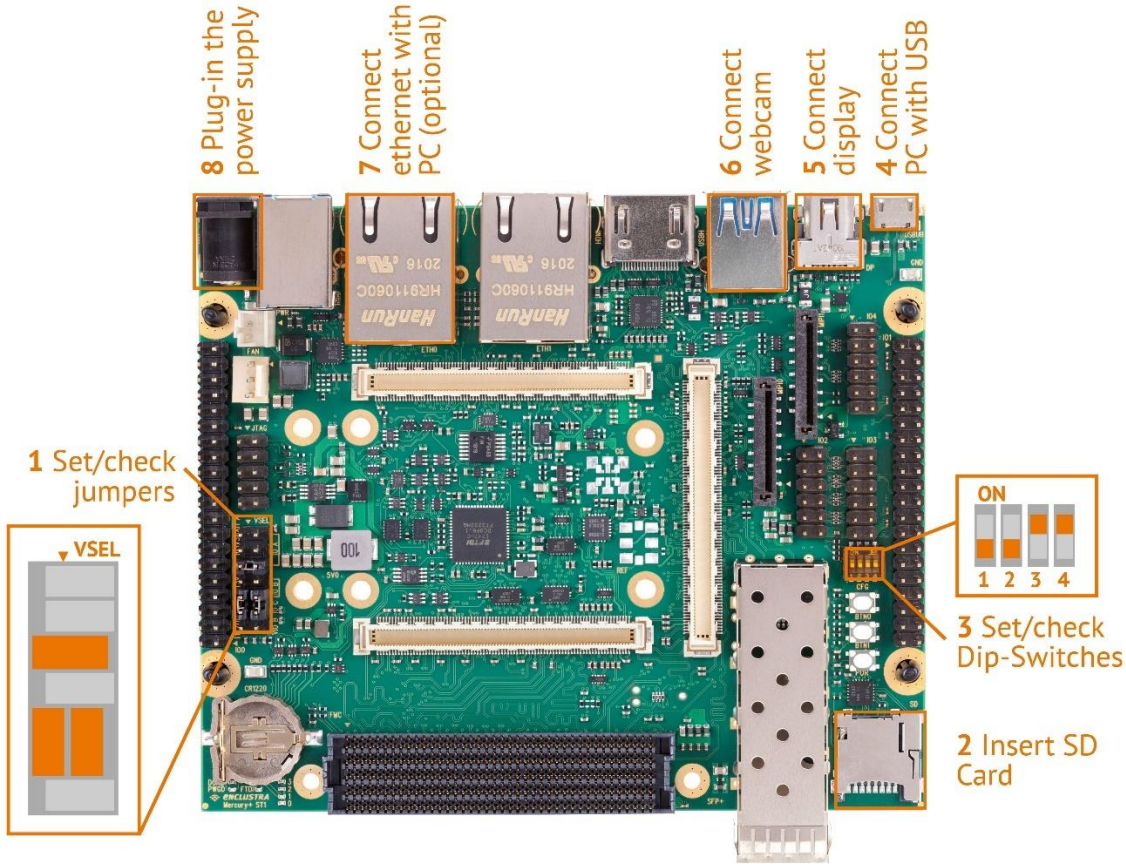
5 Running the Vitis-AI Demos

5.1 Mount the Module, Heatsink and Fan to the Base Board



5.2 Setup the Base Board

1. Follow the steps in the order indicated in the image below
2. Set/check all jumpers as shown in the image below
3. Insert the prepared SD Card
    - A working SD Card is already inserted by default
4. Set/check all DIP switches as shown in the image below
5. Use the Micro USB cable to connect the base board to the computer
6. Connect a display port monitor to the ST1 base board (DP)
7. Connect the Logitech USB camera (USBH)
8. Optional: Connect the Ethernet Cable to ETH0. Use the same network as the development Linux PC (or establish a point to point connection by assigning a fixed IP addresses)
9. Plug in the 12 V power supply



5.3 Run the Facedetect Demo

1. Power on the board
2. Open a serial terminal connection (two /dev/ttyUSBs are established, use the 2nd one e.g. /tio/ttyUSB1)
3. Login as root/root
4. Check that you can ping the board from your PC
5. Start the facedetect demo:
   `root@ST1_ME-XU5-4EV-2I-D12E:~# facedetect`

   The live video of the USB camera should be displayed on the monitor. Bounding boxes as well as statistics are overlaid on the live stream.

6. Optional: View the stream on your PC via network using the following GStreamer pipeline:
   `gst-launch-1.0 tcpclientsrc host=IP_ADDRESS_OF_THE_BOARD port=7001 ! jpegdec ! videoconvert ! autovideosink`

   The expected output:

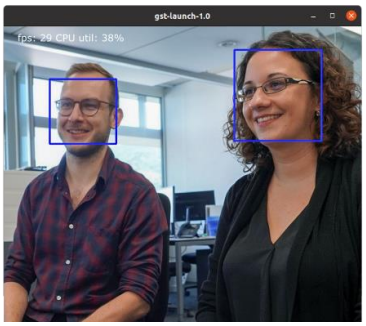


Figure 4: Expected output of the facedetect demo

5.4 Run the Resnet50 Demo

1. Power on the board
2. Open a serial terminal connection (two /dev/ttyUSBs are established, use the 2nd one e.g. /tio/ttyUSB1)
3. Login as root/root
4. Check that you can ping the board from your PC
5. Run the resnet50 example:
   root@ST1_ME-XU5-4EV-2I-D12E:~# resnet50
1. Optional: View the stream on your PC via network using the following GStreamer pipeline:
   gst-launch-1.0 tcpclientsrc host=IP_ADDRESS_OF_THE_BOARD port=7001 ! jpegdec ! videoconvert ! autovideosink

   The expected output:

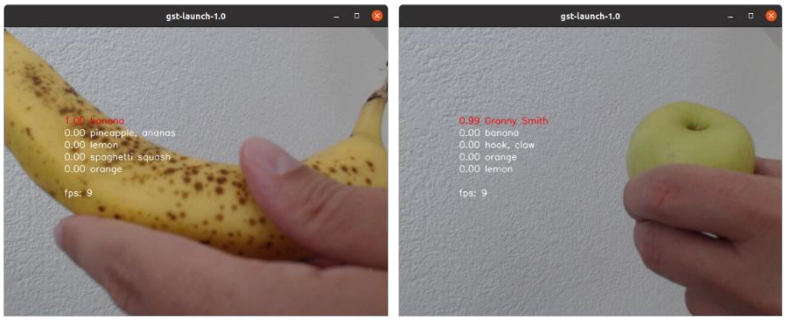


Figure 5: Expected output Resnet50

Figures

Tables

6 References

1. Xilinx Vitis AI User guide

   https://www.xilinx.com/support/documentation/sw_manuals/vitis_ai/1_3/ug1414-vitis-ai.pdf

2. Xilinx Vitis AI Repository

   https://github.com/Xilinx/Vitis-AI

3. Enclustra Mercury XU5 + ST1 User guide

   https://github.com/enclustra/Mercury_XU5_ST1_Reference_Design/blob/master/reference_design/doc/Mercury_XU5_ST1.pdf

4. Enclustra Mercury XU5 + ST1 reference design

   https://github.com/enclustra/Mercury_XU5_ST1_Reference_Design

5. Xilinx Vitis-AI Vivado DPU TRD

   https://github.com/Xilinx/Vitis-AI/blob/master/dsa/DPU-TRD/prj/Vivado/

6. Xilinx DPU Product Guide

   https://www.xilinx.com/support/documentation/ip_documentation/dpu/v3_3/pg338-dpu.pdf

7. Xilinx PetaLinux User Guide

   https://www.xilinx.com/support/documentation/sw_manuals/xilinx2020_2/ug1144-petalinux-tools-reference-guide.pdf

8. Enclustra Mercury XU5 Design-In Kit Sources

   http://enclustra.com/design-in-kit#Enclustra_Mercury_XU5_Design-In_Kit_Sources